

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Organización Empresarial

TRABAJO PRÁCTICO INTEGRADOR

Automatización del Proceso de Solicitud de Vacaciones
mediante Chatbot (Simulador de Consola — Python)

Alumno: **Ignacio Merlo**

Cátedra: **Organización Empresarial**

Docente titular: **Prof. Gabriela Martínez**

Repositorio: **github.com/IgnacioMerlo/Tpi-chatbot-vacaciones-IM**

Año: **2026**

ÍNDICE

1. Introducción
2. Análisis sistémico de la organización
3. Proceso actual (AS-IS) y diagrama BPMN
4. Proceso propuesto (TO-BE) y diagrama BPMN
5. Análisis del modelo BPMN — coherencia con el código
6. Base de datos simulada (JSON)
7. Máquina de estados del chatbot
8. Diccionario de datos
9. Casos de excepción y robustez
10. Herramientas de IA utilizadas
11. Tecnologías utilizadas y repositorio
12. Conclusión

1. INTRODUCCIÓN

TechSolutions S.A. es una empresa de desarrollo de software con más de 100 empleados. Las solicitudes de vacaciones se realizan manualmente por correo electrónico a RRHH, generando demoras y errores administrativos. Se propone un chatbot en modo consola (**chatbot.py**) que automatiza el proceso: valida el legajo del empleado en **empleados.json**, consulta el saldo disponible, registra la solicitud en **solicitudes.json** y responde de forma inmediata. Solicitudes de hasta 15 días se aprueban automáticamente; las que superan ese límite se derivan al jefe de área. El sistema no requiere dependencias externas a Python 3.

2. ANÁLISIS SISTÉMICO DE LA ORGANIZACIÓN

Según la Teoría General de Sistemas (TGS), TechSolutions S.A. es un sistema abierto, socio-técnico y dinámico. El proceso de vacaciones es un subsistema administrativo:

Componente	Descripción
Entradas	Solicitudes de empleados, legajo, saldo de días, políticas de RRHH.
Procesos	Recepción, validación de identidad, consulta de saldo, registro y notificación.
Salidas	Solicitudes aprobadas o rechazadas, registros en solicitudes.json, notificaciones.
Retroalimentación	Mensajes del bot: aprobación, rechazo o solicitud de corrección de datos.
Clasificación	Sistema abierto, socio-técnico, dinámico, orientado a objetivos organizacionales.

Tabla 1. Análisis sistémico del proceso según TGS.

3. PROCESO ACTUAL (AS-IS) Y DIAGRAMA BPMN

El empleado envía un correo a RRHH. El personal consulta una planilla manual, verifica el saldo y responde días después. El proceso es lento y propenso a errores. **Flujo:** (1) Empleado redacta correo → (2) RRHH recibe → (3) Consulta planilla → (4) Verifica saldo → (5) Aprueba/rechaza → (6) Responde al empleado.

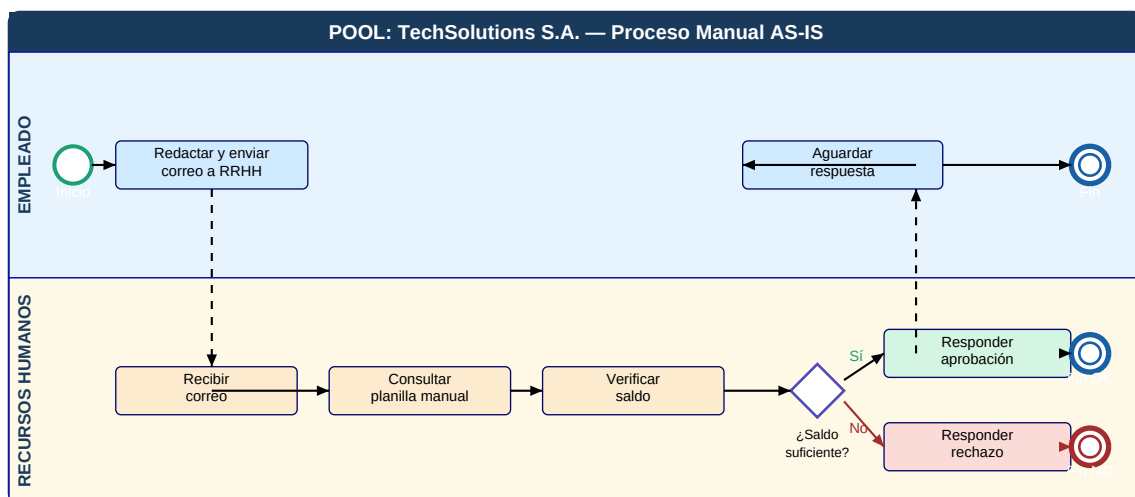


Figura 1. Diagrama BPMN 2.0 AS-IS — 2 carriles: Empleado y RRHH. Repositorio: github.com/IgnacioMerlo/Tpi-chatbot-vacaciones-IM

4. PROCESO PROPUESTO (TO-BE) Y DIAGRAMA BPMN

El chatbot (**chatbot.py**) automatiza las tareas de RRHH. **Flujo:** (1) Empleado ejecuta chatbot.py → (2) Bot pide legajo → (3) Sistema valida en empleados.json [GW1] → (4) Consulta saldo → (5) Empleado ingresa días → (6) Sistema valida formato [GW2] y saldo [GW3] → (7) Empleado confirma → (8) Si ≤ 15 días aprueba automáticamente, si > 15 deriva a RRHH [GW4] → (9)

Registra en solicitudes.json.

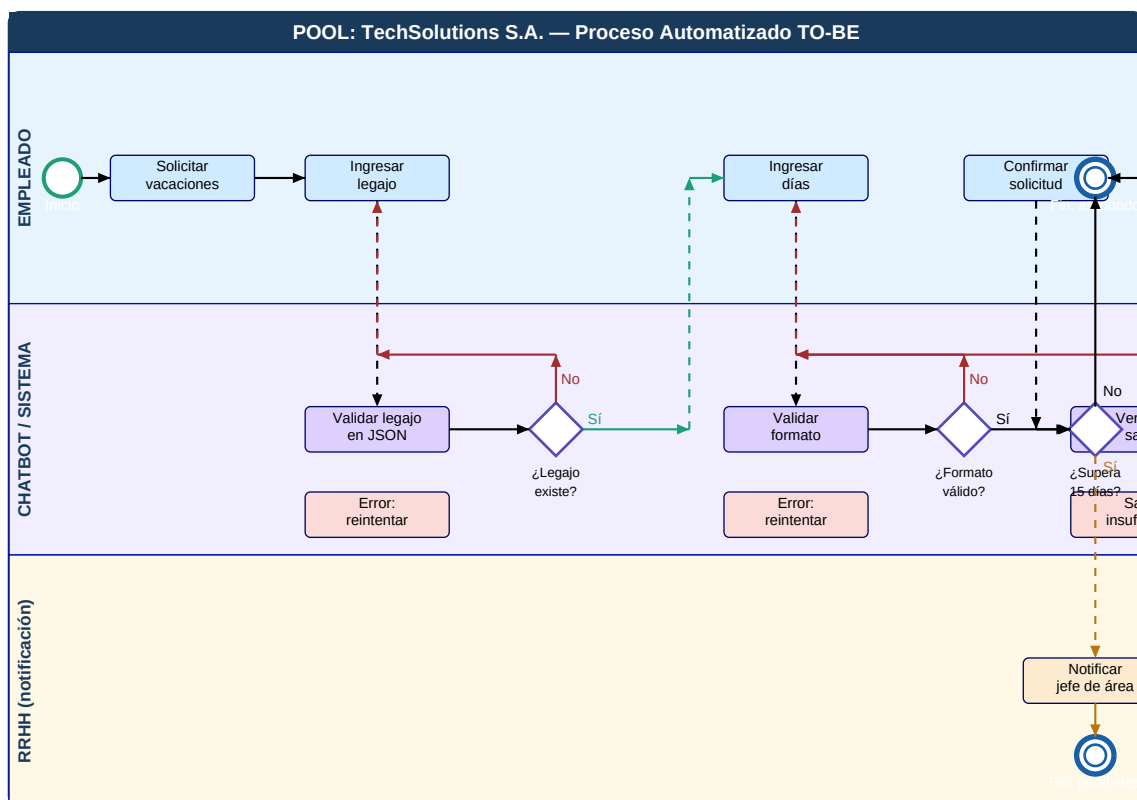


Figura 2. Diagrama BPMN 2.0 TO-BE — 3 carriles, 4 compuertas XOR, bucles de reintento. Repositorio: github.com/IgnacioMerlo/Tpi-chatbot-vacaciones-IM

5. ANÁLISIS DEL MODELO BPMN — COHERENCIA CON EL CÓDIGO

Gatewa y	Condición BPMN	Código en chatbot.py	Camino No
GW1	¿Legajo existe en empleados.json?	<code>obtener_empleado(legajo) is None</code>	Bucle reintento, máx. 3
GW2	¿Dato es número entero > 0?	<code>entrada.lstrip('-').isdigit() y dias > 0</code>	Bucle reintento, máx. 5
GW3	¿Días solicitados ≤ saldo disponible?	<code>dias > saldo → saldo insuficiente</code>	Bucle hasta /cancelar
GW4	¿Días solicitados > 15?	<code>requiere_rrhh = dias_pedidos > 15</code>	No aplica (bifurcación)

Tabla 2. Correspondencia entre compuertas BPMN y lógica en chatbot.py.

- **Pool:** TechSolutions S.A. — entidad contenedora del proceso.
- **Lanes:** Empleado, Chatbot/Sistema y RRHH con responsabilidades delimitadas.
- **Tareas de usuario:** ingresar legajo, cantidad de días y confirmar solicitud.
- **Tareas de servicio:** validar legajo, consultar saldo, validar formato, registrar en JSON.
- **Bucles de reintento:** los caminos de error retornan al estado anterior; no terminan abruptamente.

6. BASE DE DATOS SIMULADA (JSON)

El sistema usa dos archivos JSON. **empleados.json** almacena el saldo de días; **solicitudes.json** guarda el historial. Las funciones **cargar_empleados()**, **guardar_empleados()**, **cargar_solicitudes()** y **guardar_solicitudes()** en **chatbot.py** gestionan lectura y escritura. Los días

se descuentan automáticamente al registrar una solicitud aprobada.

legajo	nombre	area	dias_disponibles	estado
1001	Juan Pérez	Desarrollo	15	Activo
1002	María Gómez	QA	10	Activo
1003	Pedro Fernández	DevOps	0	Activo
1004	Ana López	RRHH	20	Activo
1005	Carlos Ruiz	Soporte	5	Activo

Tabla 3. Contenido de `empleados.json` — empleados de prueba con saldo de vacaciones.

Cada registro en `solicitudes.json` contiene: **id_solicitud** (SOL-XXXX), **legajo**, **nombre**, **dias_solicitados**, **fecha_solicitud** (ISO), **resultado** (APROBADO / PENDIENTE_RRHH) y **requiere_rrhh** (Sí/No).

7. MÁQUINA DE ESTADOS DEL CHATBOT

La función `correr_flujo()` implementa la máquina de estados. Cada bloque `while True` corresponde a un estado que no avanza hasta recibir una entrada válida:

Estado	Descripción	Transición
INICIO	Muestra encabezado y empleados de prueba	→ ESPERANDO_LEGajo
ESPERANDO_LEGajo	Pide número de legajo — GW1	→ ESPERANDO_CANT_DIAS / bucle / FIN
ESPERANDO_CANT_DIAS	Pide cantidad de días — GW2 y GW3	→ CONFIRMANDO / bucle
CONFIRMANDO	Muestra resumen, pide s/n	→ REGISTRANDO / FIN
REGISTRANDO	Guarda en JSON, descuenta días del empleado	→ FIN_APROBADO / FIN_PENDIENTE
FIN_APROBADO	Solicitud aprobada automáticamente (≤ 15 d)	—
FIN_PENDIENTE_RRHH	Derivado a jefe de área (> 15 días)	—
FIN_RECHAZADO	Cancelación o bloqueo por reintentos	—

Tabla 4. Máquina de estados finitos implementada en `correr_flujo()` de `chatbot.py`.

8. DICCIONARIO DE DATOS

Variable	Tipo	Descripción	Ejemplo
legajo	Entero	Identificador único del empleado	1002
nombre	String	Nombre completo del empleado	María Gómez
area	String	Área organizacional	QA
dias_disponibles	Entero	Saldo de días disponibles en <code>empleados.json</code>	10
dias_pedidos	Entero	Días solicitados en la sesión actual	5
int_legajo	Entero	Contador de reintentos de legajo (máx. 3)	1
int_dias	Entero	Contador de reintentos de días (máx. 5)	2
requiere_rrhh	Boolean	True si <code>dias_pedidos > 15</code>	False
resultado	String	APROBADO o PENDIENTE_RRHH	APROBADO

Variable	Tipo	Descripción	Ejemplo
id_sol	String	ID generado al registrar (SOL-XXXX)	SOL-0001
fecha_solicitud	String	Fecha ISO del registro — date.today()	2026-06-19

Tabla 5. Diccionario de datos — variables de chatbot.py y campos de los archivos JSON.

9. CASOS DE EXCEPCIÓN Y ROBUSTEZ DEL SISTEMA

El sistema contempla todos los escenarios de error (Unhappy Path) implementados en **chatbot.py**. En todos los casos la máquina de estados no avanza al siguiente estado sino que retorna al actual con un mensaje descriptivo.

9.1 Legajo inexistente

Si el legajo no existe en `empleados.json`, el sistema incrementa **int_legajo** y retorna al estado `ESPERANDO_LEGAJ`O. Ante 3 intentos fallidos (`MAX_INT_LEGAJ`O = 3), bloquea la sesión.

"El legajo 9999 no está registrado en el sistema. (Intento 2 de 3)"

9.2 Saldo insuficiente

Si `dias_pedidos` supera `dias_disponibles`, el bot informa el saldo real y retorna a `ESPERANDO_CANT_DIAS`. El bucle continúa hasta recibir un valor válido o `/cancelar`.

"Pediste 12 días pero tenés 5 disponibles. Ingresá una cantidad ≤ 5 o /cancelar."

9.3 Datos no numéricos — bucle de reintento

La validación `entrada.lstrip('-').isdigit()` captura texto libre, caracteres especiales y cadenas vacías. El sistema no avanza. Máximo 5 reintentos (`MAX_INT_DIAS` = 5).

Entrada inválida	Estado	Respuesta del sistema
"abc", "juan", "--"	ESPERANDO_LEGAJ O	"El legajo debe ser un número entero. Ejemplo: 1002. (Intento N de 3)"
"muchos", "quince"	ESPERANDO_CANT_ DIAS	"La cantidad debe ser un número entero mayor a cero. Ejemplo: 5. (Intento N de 5)"
"0", "-3"	ESPERANDO_CANT_ DIAS	"La cantidad de días debe ser mayor a cero."

Tabla 6. Respuestas del sistema ante entradas no numéricas o inválidas.

9.4 Empleado sin saldo disponible

Si `dias_disponibles` == 0 (Pedro Fernández, legajo 1003), el sistema informa y finaliza la sesión sin continuar. El empleado debe contactarse con RRHH.

"No tenés días disponibles para vacaciones. Comunícate con RRHH."

9.5 Comandos globales — `/cancelar` y `/start`

`/cancelar` termina la operación en cualquier estado. `/start` reinicia el flujo completo invocando recursivamente a `correr_flujo()`, simulando el reinicio de sesión del bot real de Telegram.

Escenario	Variable afectada	Comportamiento	Límite
Legajo inexistente	int_legajo	Bucle en <code>ESPERANDO_LEGAJ</code> O	3 intentos
Saldo insuficiente	dias_pedidos vs saldo	Bucle en <code>ESPERANDO_CANT_DIAS</code>	<code>/cancelar</code>
Dato no numérico legajo	int_legajo	Bucle + contador	3 intentos
Dato no numérico días	int_dias	Bucle + contador	5 intentos
Saldo = 0	dias_disponibles	Fin inmediato con mensaje	Sin bucle
<code>/cancelar</code>	—	Fin de sesión inmediato	—
<code>/start</code>	—	Reinicio completo del flujo	—

Tabla 7. Resumen de escenarios de excepción y comportamiento del sistema.

10. HERRAMIENTAS DE IA UTILIZADAS

Herramienta	Aplicación en el proyecto	Prompt utilizado
Claude (Anthropic)	Corrección del BPMN TO-BE (carriles, gateways con bucles), generación de chatbot.py adaptado a JSON y revisión de robustez.	"Corregí el BPMN TO-BE: delimitá Pools/Lanes estrictos y asegurate de que los gateways tengan bucles de reintento en lugar de terminar abruptamente."
ChatGPT (OpenAI)	Propuesta inicial del diagrama BPMN y análisis preliminar de la consigna del TPI.	"Diseñar chatbot de vacaciones con BPMN 2.0, validación de legajo y manejo de errores."

Tabla 8. Herramientas de IA utilizadas, aplicación y prompts empleados.

El análisis final, las decisiones de diseño, las correcciones conceptuales y la redacción definitiva fueron realizados por el autor del proyecto. Las herramientas de IA se utilizaron únicamente como apoyo y punto de partida.

11. TECNOLOGÍAS UTILIZADAS Y REPOSITORIO

Componente	Tecnología	Detalle
Lenguaje	Python 3.x	Sin dependencias externas adicionales a la librería estándar
Módulos usados	json, os, datetime	Incluidos en Python 3 — no requieren instalación adicional
Base de datos	JSON (empleados.json + solicitudes.json)	Lectura y escritura con json.load() / json.dump()
Entorno	VS Code	Terminal integrada — ejecutar: python chatbot.py
Modelado BPMN	Draw.io / diagrams.net	Herramienta gratuita con soporte completo BPMN 2.0
Repositorio	GitHub	Control de versiones, README.md y acceso público

Tabla 9. Stack tecnológico del proyecto. Repositorio: github.com/IgnacioMerlo/Tpi-chatbot-vacaciones-IM

12. CONCLUSIÓN

La automatización del proceso de solicitud de vacaciones mediante un simulador de consola en Python permite reducir tiempos de gestión, eliminar errores administrativos y demostrar de forma clara la lógica de negocio modelada en BPMN 2.0. El sistema implementa exactamente lo que el modelo define: tres carriles con responsabilidades delimitadas, cuatro compuertas de decisión con correspondencia directa en el código (**chatbot.py**), bucles de reintento conversacional ante cualquier entrada inválida y una máquina de estados que garantiza la coherencia del flujo. La persistencia se logra con dos archivos JSON que el sistema lee y actualiza en cada ejecución. Este trabajo integró organización empresarial, análisis de procesos y programación, demostrando cómo la tecnología aporta valor operativo real mediante la automatización de tareas administrativas repetitivas.